



SigmaStar Display

User Development Guide



© 2020 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.



{SigmaStar Display}

{Smart Display}

{SSD201+1.0}

REVISION HISTORY

Revision No.	Description	Date
0.1	ÿ Initial release	06/21/2019
0.2	ÿUpdate	31/10/2019
	ÿ	



{SigmaStar Display}
{Smart Display}
{SSD201+1.0}

TABLE OF CONTENTS

REVISION HISTORYi

TABLE OF CONTENTS..... ii

1. Familiarize yourself with the SDK components 1

 1.1. Download SDK 1

 1.2. Build the build environment..... 1

 1.2.1 Installing a Linux server 2

 1.3. Compiling the SDK. 3

 1.3.1 Unzip the code 4

 1.3.2 Compile boot.....4

 1.3.3 Compile the kernel 4

 1.3.4 Compile SDK(ALKAID) 4

 1.4. Programming..... 5

 1.4.1 Burning code by uboot 5

 1.4.2 Burning uboot by ISP Tool..... 6

1.5. How to Customize the Flash Partition..... Error! Bookmark not defined .

 1.5.1 Spinor Partition Information Adjustment..... Error! Bookmark not defined.

1.6. Detailed explanation of memory usage.....12

1.7. MMAP adjustment method14

 1.7.1 mmap.ini Description14

 1.7.2 Adjust Layout in mmap.ini14

1.8. Precautions15

1. Familiar with SDK components

1.1. Download SDK

From the ftp provided by SStar, you can down to the following resources:

 boot-TAKOYAKI-ALPHA001-0606.tar.gz
 gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi.tar.gz
 kernel-TAKOYAKI-ALPHA001-0606.tar.gz
 project-TAKOYAKI-ALPHA001-0606.tar.gz
 sdk-TAKOYAKI-ALPHA001-0606.tar.gz

Figure 1: Figure 1-1 SDK

package boot: Uboot source code kernel: kernel

source code, version: 4.9.84 project: part of

compiling and making image, including release header files and lib library sdk: part of test

code gcc-xxx: part of toolchain , glibc version provides 8.2.1 64bit [1.2. Build environment](#)

Usually we develop and debug in the form of cross-compilation, that is, in the form of "host machine + target machine". And the connection between the host machine and the target machine

They generally use serial connection or network connection, as shown in the following figure:



Figure 2: Figure 1-2 Debug connection method

Note: In addition to **debugging** , the **debug tool** we provide can also read registers and program empty **flash** through **PC** tools. If it is only

for debugging, you can use the serial device to correctly connect the debugging line sequence (con3). Please align the line sequence with

the schematic diagram. If you need to read Register or burn bare Flash, you need Debug Tool. tool Buy according to demand [https://](https://item.taobao.com/item.htm?id=15437784110)

item.taobao.com/item.htm?id=15437784110



1.2.1 Install Linux server

Ubuntu 16.04 is recommended.

Since the compilation of sdk requires some lib&tools, the installation process will be introduced here.

1.2.1.1. Install ubuntu 16.04 server

1. Download the server version of 1604: <http://www.ubuntu.org.cn/download/server>, choose to download the 1604 server version 2.
- Install VMware Workstation 3. Select the 1604 iso, load and install it 4. Easy installation Click Finish directly, the system will automatically install 5. Install VMtools, which is used to copy and share the windows directory to ubuntu, refer to:

<https://www.cnblogs.com/huangjianxin/p/6343881.html>

Ubuntu has been successfully installed so far, and then install some environments required for development.

1.2.1.2. Install lib tool & required for compilation

Next, install the corresponding compilation

tools, etc.: 1. Newly install ubuntu, first set the root password, enter the system to set the root password: # sudo passwd, enter the password continuously, and the prompt can be successful 2. Add users, when creating a virtual machine You will be prompted to enter the user count, but if you want to use beal.wu and other similar accounts, you cannot enter it when creating ubuntu (it will prompt a format error), and the system will prompt that the account is illegal, but because you need to use beal.wu, you need to After entering the system, re-create the user, # sudo adduser beal.wu --force to create the user, and you will be prompted to enter some account information, just press Enter.

3. At this point, the account has been established, then you need to install the relevant tools and compilation environment, execute # sudo apt-get install libcurses5-dev libcursesw5-dev, then you will be prompted that beal.wu cannot execute sudoers, you need to make the following modifications: sudo chmod u+w /etc/sudoers vi /etc/sudoers; refer to root ALL=(ALL:ALL) ALL, add a line **beal.wu ALL=(ALL:ALL) ALL** sudo chmod uw /etc/sudoers and then execute sudo operation Installed, it is recommended to execute #sudo apt-get update after installation to update the source 4. Install samba for sharing linux files to windows

```
# sudo apt-get install samba samba-common # sudo
```

```
smbpasswd -a beal.wu (add a samba account for windows to access the linuxsamba directory) Modify /etc/samba/smb.conf
```

```
[beal.wu]
path=/home/beal.wu
public=yes writable=yes
```

```
valid users=beal.wu
available=yes
browseable=yes
guest ok = yes
```

In this way, samba has been configured, restart the samba service, and test accessing samba from windows: \\192.168.1.11 (your ubuntu ip)



5. Install ssh for login debugging

sudo apt-get install openssh-server; ready to use after installation

6. Some tools need to be installed for SDK compilation, otherwise

compilation will fail # sudo apt-get install libc6-dev-i386 # sudo apt-get install lib32z1 lib32ncurses5 # sudo apt-get install libuuid1:i386 # sudo apt-get install cmake # sudo apt-get install libncurses5-dev libncursesw5-dev # sudo apt install bc # sudo apt-get install xz-utils # sudo apt-get install automake # sudo apt-get install libtool # sudo apt-get install libudev-dev # sudo apt-get install pkg-config The tools && lib that need to be installed as above are necessary, because the relevant content will be used in the compilation process. I won't explain what is wrong here.

If it is wrong, you can try to check the related errors without installing and compiling first. To save trouble, you can install it once before compiling.

7. If the default sh is not bash, you need to change sh to bash:

sudo rm /bin/sh sudo

ln -s /bin/bash /bin/sh 8. Install

toolchain and copy "gcc-arm-8.2-2018.08-

x86_64-arm-linux-gnueabi.tar.gz" to /opt/ directory by yourself Define the decompression toolchain and set the toolchain to the environment variable, so as not to need to manually export each time; Add in /etc/profile: export PATH=/opt/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi/bin: \$PATH

9. At this point, the compilation is executed.

1.2.1.3. Install cross-compilation tools

We provide the compilation toolchain gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi for compiling the glibc version.

Glibc: 2.28

```
beal.wu@sigmastar:~/i2m/sdk$ /opt/ToolChain/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi/bin/arm-linux-gnueabi-gcc
Using built-in specs.
COLLECT_GCC=/opt/ToolChain/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi/bin/arm-linux-gnueabi-gcc
COLLECT_LTO_WRAPPER=/opt/ToolChain/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi/bin/../libexec/gcc/arm-linux-gnueabi
Target: arm-linux-gnueabi
Configured with: /tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/src/gcc/config
ith-sysroot=/arm-linux-gnueabi/libc --with-build-sysroot=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64
ueabi/install/arm-linux-gnueabi/libc --enable-gnu-indirect-function --enable-shared --disable-libssp --disable-li
ing=release --enable-languages=c,c++,fortran --with-gmp=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--a
bihf/host-tools --with-mpfr=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/bui
tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/build-arm-linux-gnueabi/host-t
he6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/build-arm-linux-gnueabi/host-tools --with-arch=armv7-a --wi
7-a --with-pkgversion='GNU Toolchain for the A-profile Architecture 8.2-2018-08 (arm-rel-8.23)'
Thread model: posix
gcc version 8.2.1 20180802 (GNU Toolchain for the A-profile Architecture 8.2-2018-08 (arm-rel-8.23))
beal.wu@sigmastar:~/i2m/sdk$
```

1.3. Compiling the SDK

SSD20x supports two startup modes: nor flash and spi nand flash, so the compilation in the SDK is differentiated through different configuration files To achieve, the SDK contains the sourcecode of uboot and kernel, and can generate burnable images by packaging the script in the project.

Both Spinor/Spinand can work normally. The Display demo board is equipped with Spinor flash. Customers can layout by themselves.

The Display demo board of SSD20x needs to be replaced with Spinand for debugging, please contact us for hardware.



{SigmaStar Display}

{Smart Display}

{SSD201+1.0}

1.3.1 Unzip the code

Obtain the respective SDK development packages from FTP, and obtain the sourcecode, the format is as follows: (screenshot

name is for reference only) tar zxvf boot-TAKOYAKI-ALPHA001-0606.tar.gz

...

tar zxvf sdk-TAKOYAKI-ALPHA001-0606.tar.gz 1.3.2

Compile boot

ÿ SPI-NOR package

```
#declare -x ARCH="arm"

#declare -x CROSS_COMPILE="arm-linux-gnueabihf-" #make
infinity2m_defconfig; //spinor #make infinity2m_spinand_defconfig; //
Spinand #make menuconfig; //modify && save #make clean;
```

```
#make
```

ÿ Get image

```
# cp u-boot.xz.img.bin ${ your_release_path } //spinor # cp u-
boot_spinand.xz.img.bin ${ your_release_path } //Spinand 1.3.3 compile kernel
```

ÿ Kernel

CHIP	Glibc compiler	Ulibc compiler	Kernel make config
SSD20x spinor	arm-linux-gnueabihf- NA		infinity2m_ssc011a_s01a_minigui_defconfig
SSD20x spinand	arm-linux-gnueabihf- NA		infinity2m_spinand_ssc011a_s01a_minigui_defconfig

Note: Please refer to the table and compile the corresponding version of your chip. The first version of Ulibc SDK is not supported for the time being and will be improved in the future.

```
# declare -x ARCH="arm"

# declare -x CROSS_COMPILE=" arm-linux-gnueabihf-" # make
infinity2m_ssc011a_s01a_minigui_defconfig //SSD20x spinor # make
infinity2m_spinand_ssc011a_s01a_minigui_defconfig //SSD20x Spinand # make menuconfig; //
modify && save # make clean; # make
```

ÿ Get image # cp

```
arch/arm/boot/ulmage.xz ${ your_release_path } Note: If you want
```

to update uboot and kernel, you need to compile the project first, and then copy the bins of uboot and kernel to /yourpath/xxx/image/output/ In image , the kernel is ulmage.xz -> kernel 1.3.4 Compile SDK (ALKAID)

ÿ Build SDK package

CHIP	Glibc	Ulibc
QFN128 64M: SSD201 Nor	nor.glibc-squashfs.011a.64	
QFN128 64M: SSD201 Nand spinand.glibc	011a.64	



QFN128 128M: SSD202D Nor	<code>nor.glibc-squashfs.011a.128</code>	
QFN128 128M: SSD202D Nand	<code>spinand.glibc.011a.128</code>	

Note: Please refer to the table and compile the corresponding

version of your chip. `# cd ${your path}/project 8.2.1:s`

```
# ./setup_config.sh ./configs/nvr/i2m/8.2.1/nor.glibc-squashfs.011a.64 //SSD20x Spinor # ./setup_config.sh ./configs/
nvr/i2m/8.2.1/spinand.glibc.011a.64 //SSD20x Spinand # make clean; make image # cd ${Alkaid}/project/
image/output/images
```

1.4.

Burning 1.4.1 Burning code by uboot

Run **tfpt (FTP server)** on PC Step1. Use the

tfpt tool to point to the image path: `SDK\project\image\output\images\`, and select the correct network card.

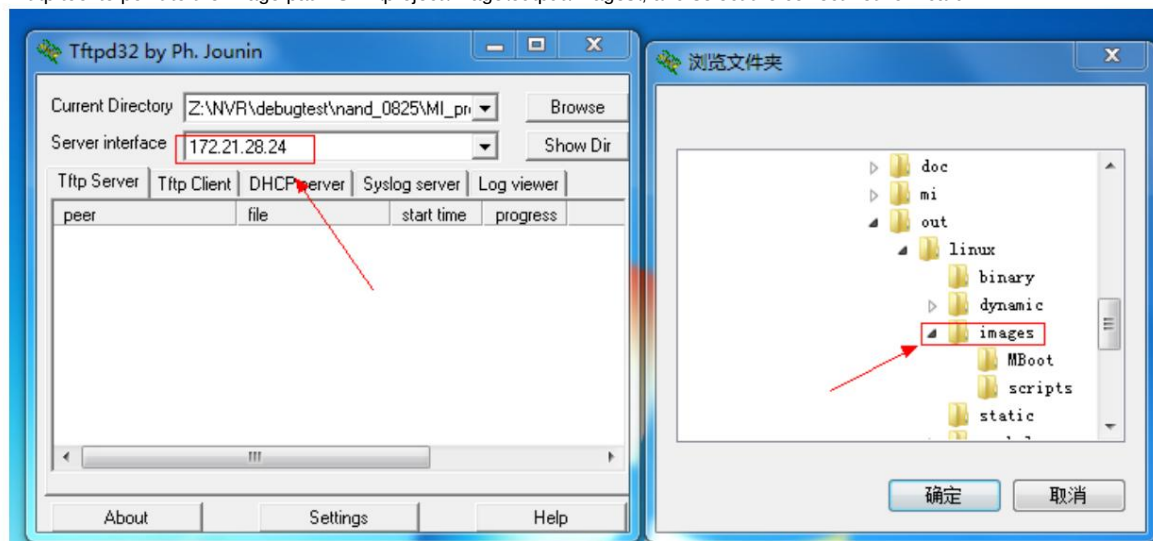


Figure 5: Figure 1-2 Step

2. Connect the network port of the board to the PC side, connect the debug serial port tool to the PC side, and check the correctness of the connection.

Run **tfpt (FTP Client)** on Demo Board Step 1. Power on

the board, press and hold Enter to enter the bootloader command line. Mstar's boot loader.

Please set the IP for the first burning: (unless the ip setting is changed or

the flash is replaced) `# setenv gatewayip 192.168.1.1 # setenv ipaddr`

`192.168.1.127`

//Set the IP used by the FTP Client (EVB board)

`# setenv netmask 255.255.255.0`

`# setenv serverip 192.168.1.100`

//Set the IP of the FTP server (PC)

`# saveenv`

Note:

1. In order to ensure smooth programming, please ensure that the PC and the development board are in the same

network segment. 2. Please use static method to assign IP fixedly. Prevent ip address jumping during programming.



3. You can also use an independent network card to directly connect the PC to the development board, fix the network ip address of the network card, and set the development board according to the above method.

ÿ Execute the following command under UBOOT console to automatically burn through ethernet.

estar (OR: estar auto_update.txt, you can also burn one of the partitions:)

1.4.2 Burning uboot by ISP Tool

Burn naked flash

This method is suitable for empty computer programming, or uboot has been damaged and cannot be upgraded through uboot.

1.4.2.1. SPI-NOR-Flash

ÿ Default Partition layout

No	range	size
IPL	0x00000000-0x00010000	64KB
IPL_CUST	0x00010000-0x00020000	64KB
MXPT	0x00020000-0x00030000	64KB
UBOOT	0x00030000-0x0004F000	124KB
UBOOT_ENV	0x0004F000-0x00050000	4KB
BOOT	0x00000000-0x00050000	256KB
KERNEL	0x00050000-0x00250000	2048KB
rootfs	0x00250000-0x00550000	3072KB
nvrsvservice	0x00550000-0x009D0000	4800KB
customer	0x009D0000-0x00F00000	5300KB

Table 1: Table 1-1

The table is for reference only, please refer to this partition information: board/i2m/boot/nor/partition/tool/mxp_gen.c The following code segment is

Partition information.

```
i=0;
out_record("IPL", MXP_PART_TYPE_NORMAL, 0x00000000, 0x00010000, MXP_PART_STATUS_EMPTY, i++); //don't modify
out_record("IPL_CUST", MXP_PART_TYPE_NORMAL, 0x00010000, 0x00020000, MXP_PART_STATUS_EMPTY, i++); //don't modify
out_record("KEY_CUST", MXP_PART_TYPE_NORMAL, 0x00020000, 0x00030000, MXP_PART_STATUS_EMPTY, i++);
out_record("MXPT", MXP_PART_TYPE_NORMAL, 0x00030000, 0x0004F000, MXP_PART_STATUS_EMPTY, i++); //0x3000, 0x20000, 0x40000
out_record("UBOOT", MXP_PART_TYPE_NORMAL, 0x0004F000, 0x00050000, MXP_PART_STATUS_EMPTY, i++);
out_record("UBOOT_ENV", MXP_PART_TYPE_NORMAL, 0x00050000, 0x00250000, MXP_PART_STATUS_EMPTY, i++);
out_record("BOOT", (MXP_PART_TYPE_NORMAL|MXP_PART_TYPE_MTD), 0x00250000, 0x00550000, MXP_PART_STATUS_EMPTY, i++);
out_record("KERNEL", (MXP_PART_TYPE_NORMAL|MXP_PART_TYPE_MTD), 0x00550000, 0x009D0000, MXP_PART_STATUS_EMPTY, i++);
out_record("rootfs", (MXP_PART_TYPE_NORMAL|MXP_PART_TYPE_MTD), 0x009D0000, 0x00F00000, MXP_PART_STATUS_EMPTY, i++);
out_record("nvrsvservice", (MXP_PART_TYPE_NORMAL|MXP_PART_TYPE_MTD), 0x00F00000, 0x00000000, MXP_PART_STATUS_EMPTY, i++);
out_record("customer", (MXP_PART_TYPE_NORMAL|MXP_PART_TYPE_MTD), 0x00000000, 0x00000000, MXP_PART_STATUS_EMPTY, i++);
```

The partition information that has been burned in the current flash can be viewed using mxp t.list under uboot

SigmaStar # **mxp t.list**

ÿ Burning code by ISP tool

	offset	Binary drop directory
IPL.bin	0x0000	\${ALKPID}\project\image\output\images\IPL.bin
IPL_CUST.bin	0x10000	\${ALKPID}\project\image\output\images\IPL_CUST.bin
MXPT_SF.bin u-	0x20000	\${ALKPID}\project\image\output\images\MXPT_SF.bin
boot.xz.img.bin 0x30000		\${ALKPID}\project\image\output\images\u-boot.xz.img.bin

Table 2: Table 1-2

ÿ Burning Steps

ÿ Step1. Execute ISP tool. And close the **UART** terminal, otherwise it can't 'Connect' normally.

If it is under uboot, enter "debug" to close it, and enter "11111" under the system.

Note: Be sure to close the serial port connection.



{SigmaStar Display}

{Smart Display}

{SSD201+1.0}

Step2. Select the SPI tab, click 'More' and select the type as 'SPI'.

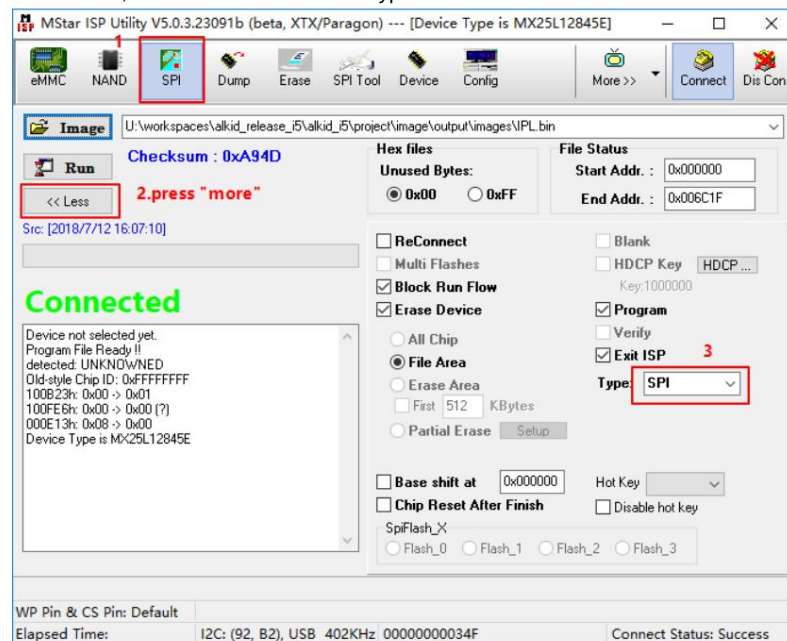


Figure 3: Figure 1-3

Step3. Load the programming file and click 'Connect'.

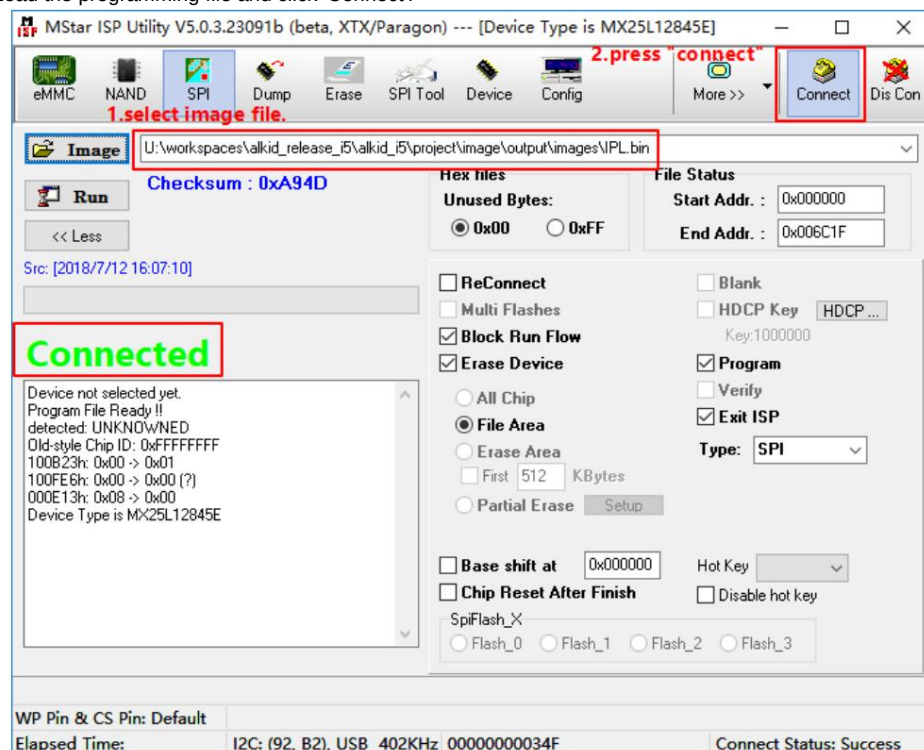


Figure 4: Figure 1-4

Step4. Load the image "IPL.bin", and click 'Run'. Note: 'erase file area' needs to be checked.



{SigmaStar Display}

{Smart Display}

{SSD201+1.0}

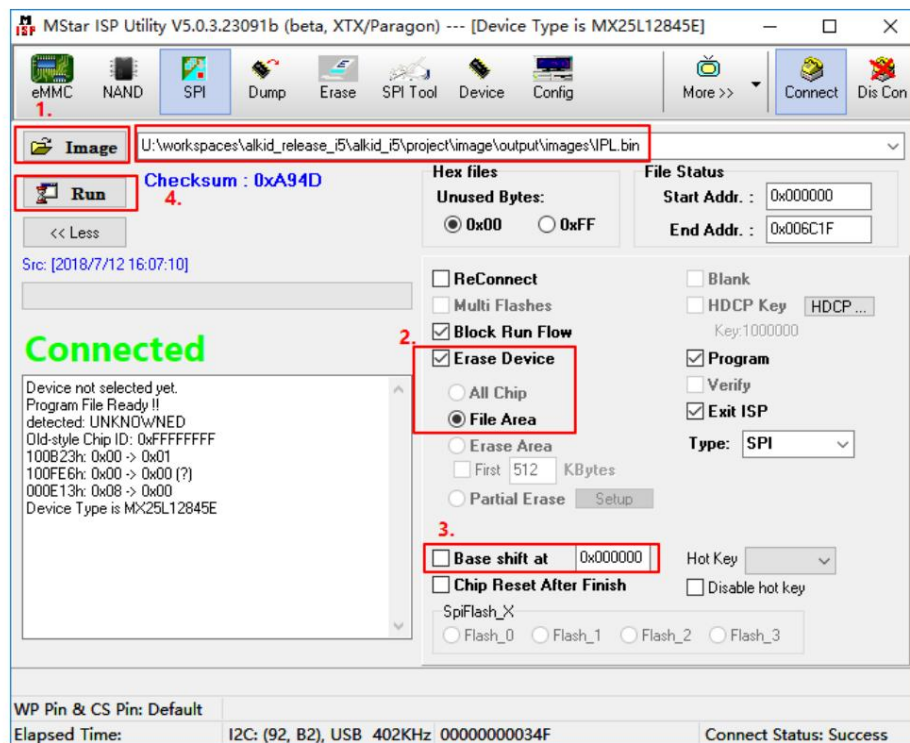


Figure 5: Figures 1-5

Step5. Load the image "IPL_CUST.bin" and cancel the 'Erase Device' option. Set 'Base shift' at 0x10000.

Note: It may change with the version, and the address of Base shift is subject to the data in Table 1-2 .

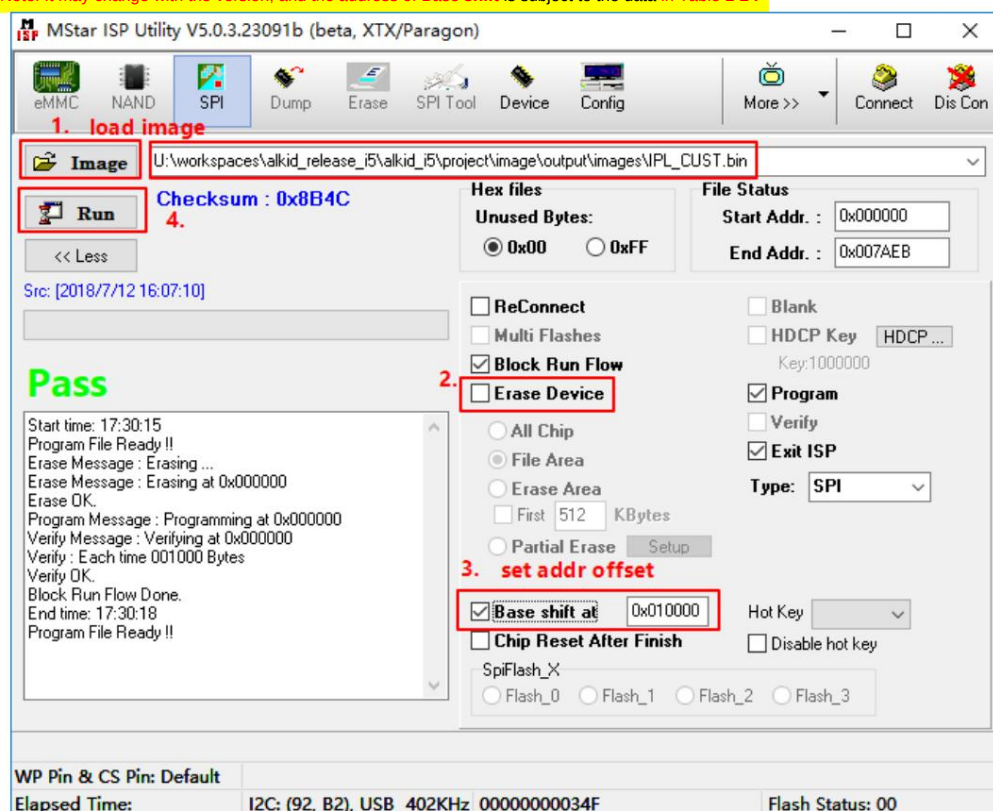


Figure 6: Figure 1-6

Step6. Load image "MXP_SF.bin", set 'Base shift' at 0x20000.



{SigmaStar Display}

{Smart Display}

{SSD201+1.0}

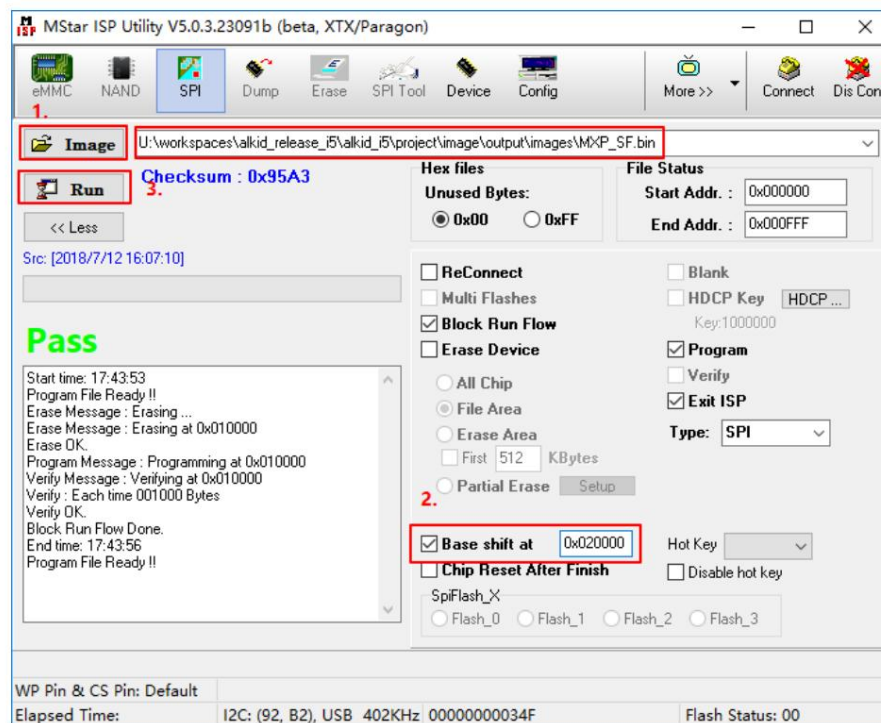


Figure 7: Figure 1-7

Step7. Load the image "u-boot.xz.img.bin" and set 'Base shift' at 0x30000.

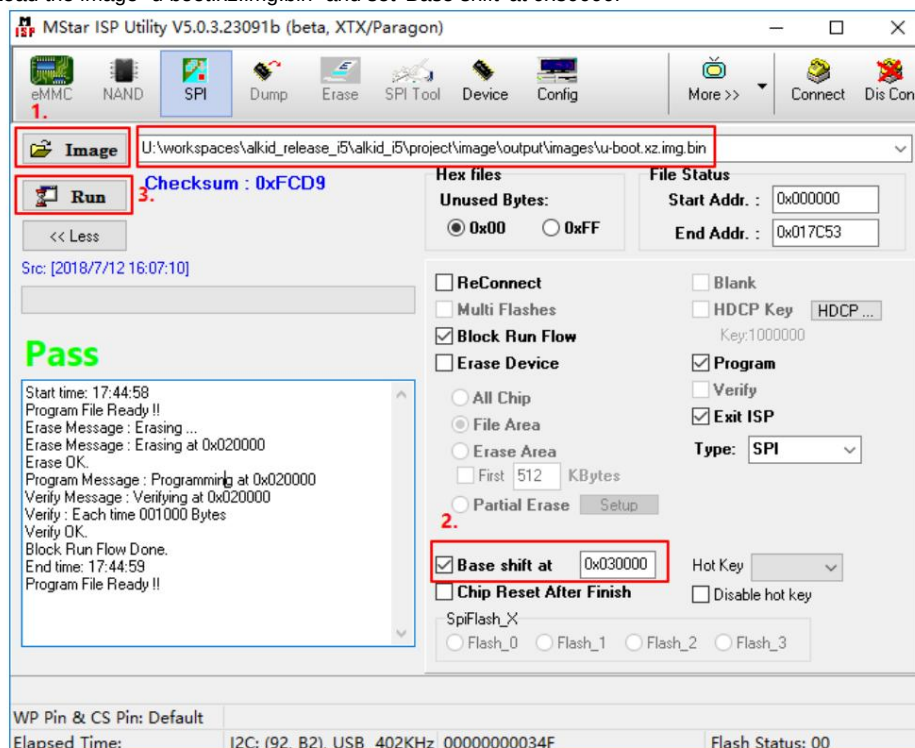


Figure 8: Figure 1-8

Step8. Restart the **Demo Board**, close the tool, and then you can burn through the network refer to 1.4.1. At this point, you should be able to enter Uboot, enter Uboot and use the TFTP burning method to burn the rest.

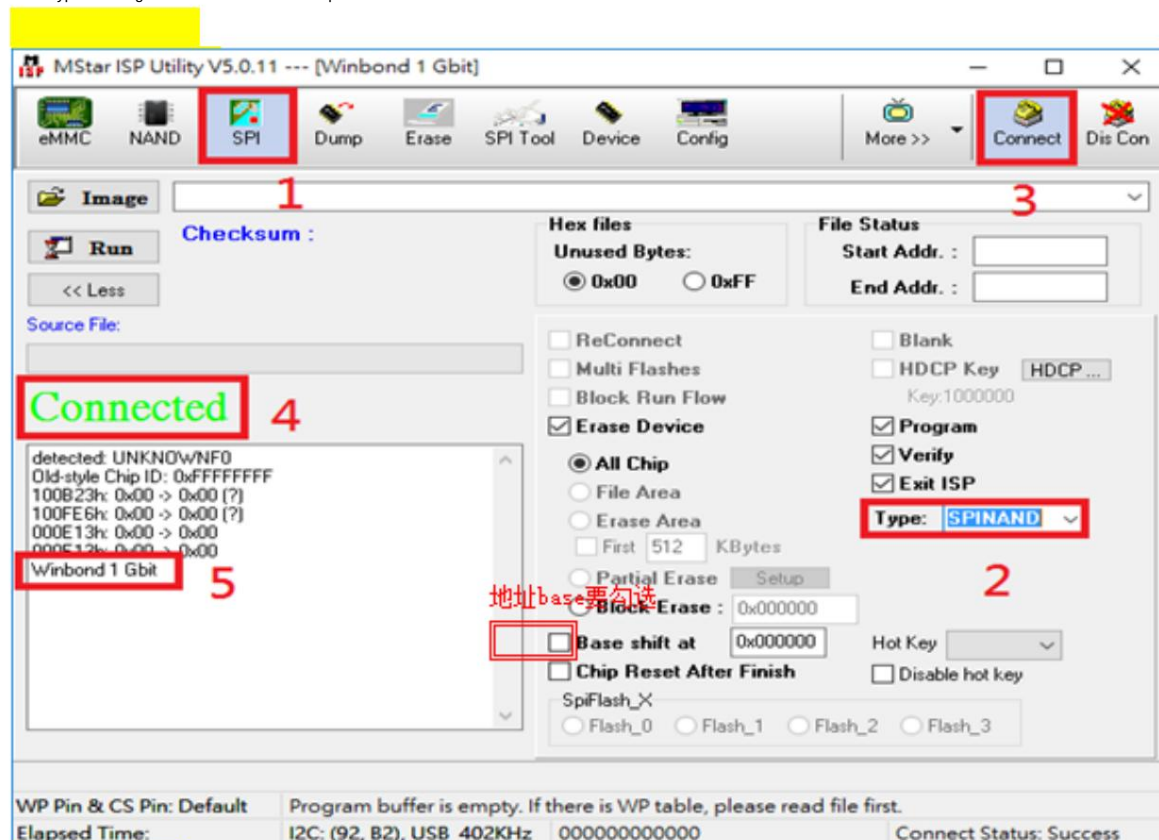


{SigmaStar Display}
{Smart Display}
{SSD201+1.0}

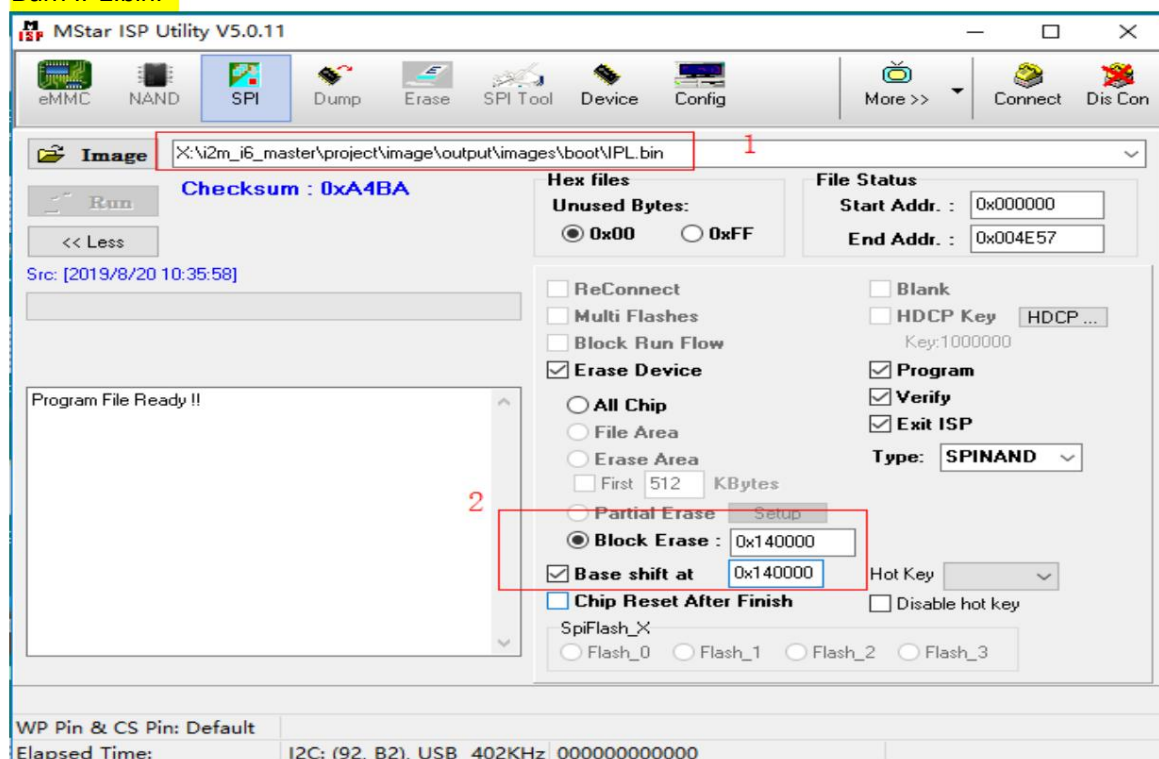
1.4.2.2. Spinand flash

The configuration of board connection is the same as that of Spinor, the difference is that SPINAND needs to be selected when choosing the

Flash type. Burning selection needs to select spinand and then connect: Burn GCIS.bin:



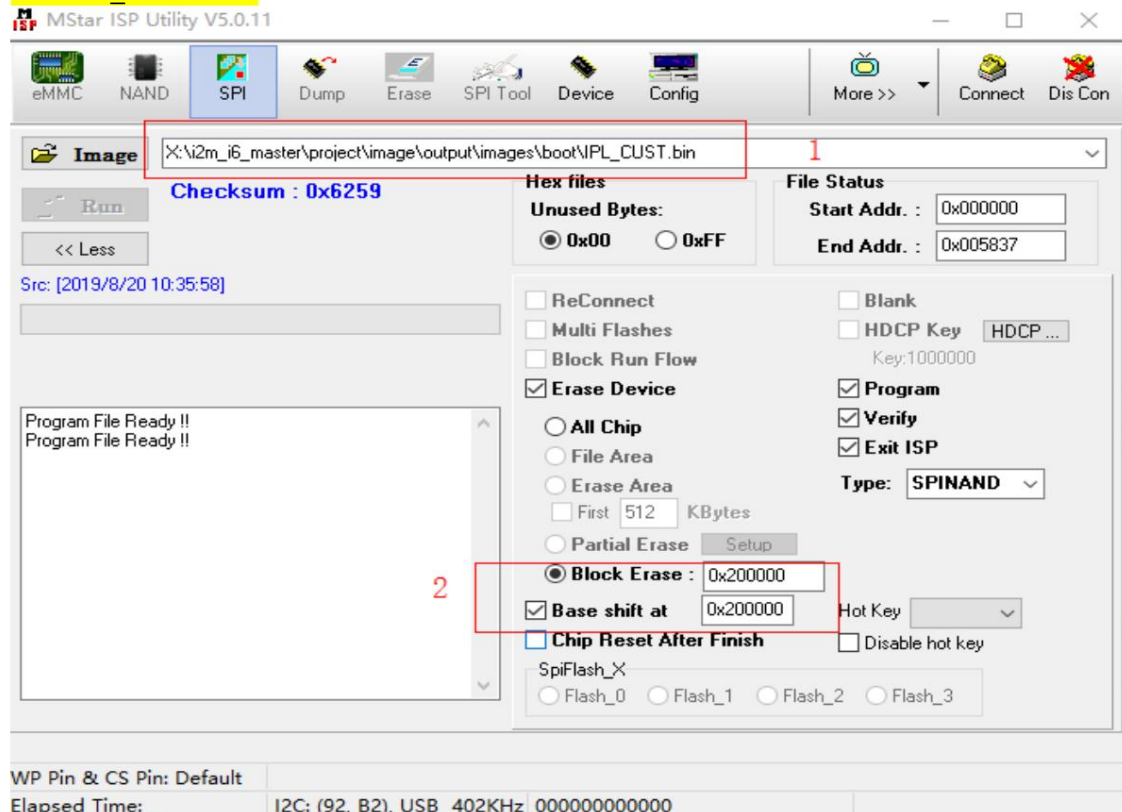
Burn IPL.bin:



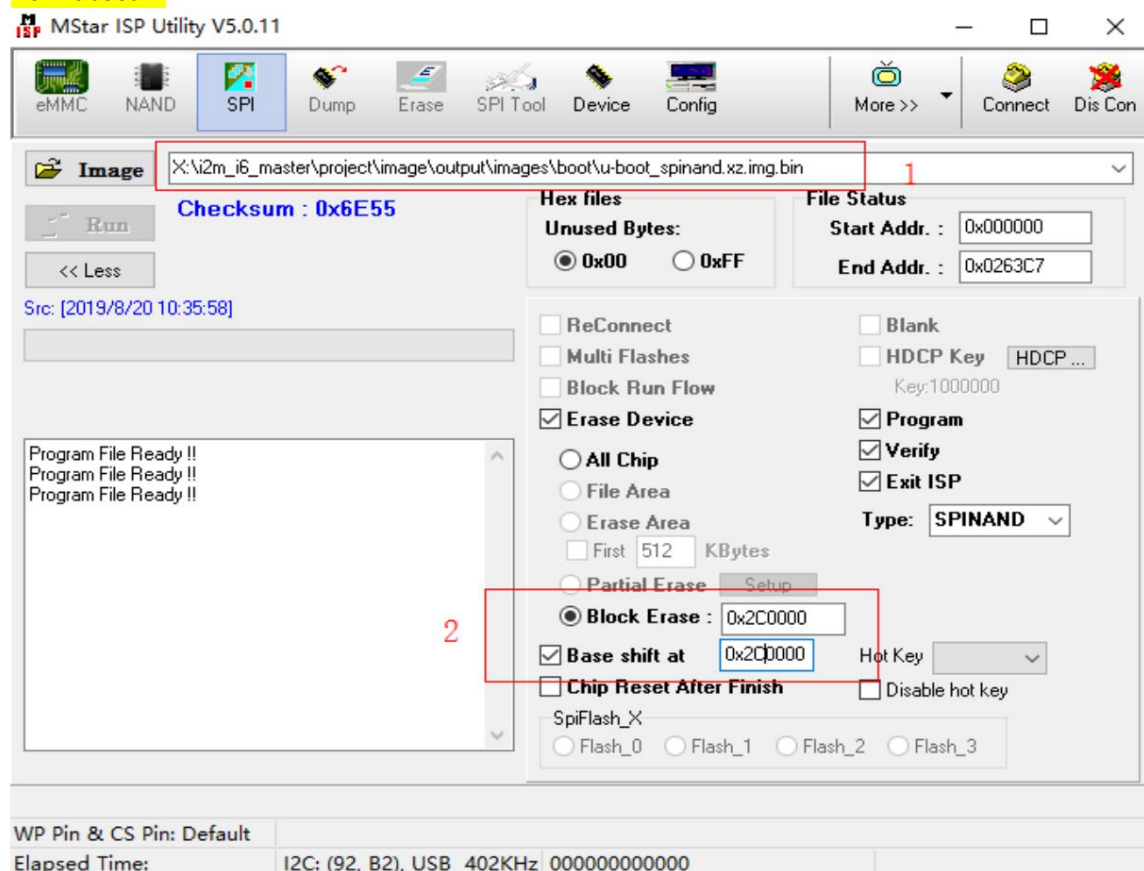


{SigmaStar Display}
 {Smart Display}
 {SSD201+1.0}

Burn IPL_CUST.bin:



Burn uboot:





{SigmaStar Display}

{Smart Display}

{SSD201+1.0}

The method of burning spinand is the same as burning spinor, but the address and file are slightly different

	Erase Device	offset	The directory where the image is located
GCIS.bin	All Chip	0x00000	yourPath/image
IP.bin	File Area	0x140000	yourPath/image
IPL_CUST.bin	File Area	0x200000	yourPath/image
u-boot_spinand.xz.img.bin	File Area After	0x2C0000	yourPath/image

burning the above files in sequence, you can enter Uboot, and then you can use the network to burn other images.

Note: When programming, it is required to stop the serial port and close the serial port tool

Stop the serial port under Uboot: enter debug

Kernel stop serial port: input 11111

When the flash is empty, how to make sure that the cpu is ok? Set the serial port buad to 38400 to see the print, not garbled

Enter uboot and execute: estar can perform tftp upgrade.

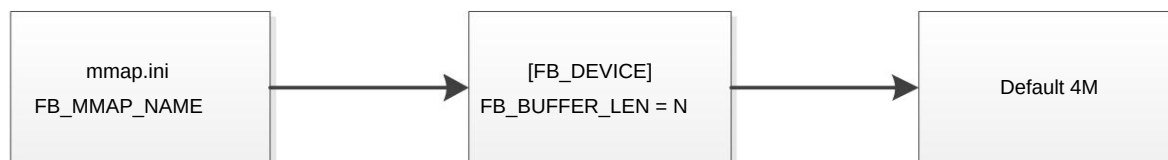
1.5. Detailed memory usage

The following table describes the situation of memory allocation: memory is equal to (LX+Layout) two parts



	composition	SIZE	use	Is it configurable? It
LX	MMA	21M (may have been modified)	Reserve a piece of memory for allocation by modules in the SDK, such as VDEC, VENC, etc. Since these require continuous physical memory, we will reserve it and hand it over to MI_SYS for management	can be modified according to the needs of customers. Normally, if only 1080P decoding and display are required, about 9M can
	Linux mem	44M	LX In addition to the reserved mma, the remaining memory is managed by linux. By default, it can be seen from /proc/meminfo that it is 40M. The reason is that the text segment of the kernel will reserve a section of memory, which is about 4M as follows (and kernel config) . Relevant, the larger the kernel, the more reserved, so you need to cut the unrelated configuration) / # cat /sys/kernel/debug/memblock/reserved 0: 0x20004000..0x20007fff 1: 0x20008240..0x203cfeb7	Can
LAYOUT AI, AO and other total		1M	Reserve some necessary small blocks of memory. Since this part also needs to be physically continuous, in order to prevent it from shredding large blocks of memory, it is given to the layout and its physical address is fixed.	Yes, requires 1M of memory that must be reserved
		64M		
Mem layout configuration	/project/board/i2m/mmap/MMAP_I2M_64M.h Use SCA.exe tool to quickly adjust /project/configs/nvr/i2m/8.2.1/			
	nor.glibc-squashfs.011a.64 sz=0x1500000			
	cma configuration, cma memory is in meminfo total, the setting method can be added in bootargs cma=2M			

The memory of the FB was also Layout in mmap.ini, but due to the flexibility and the screen resolution changes too much, it is necessary to re-layout mmap.ini frequently, so now the memory allocation of the FB is configured to be dynamically allocated from mma, but still Compatible with the previous allocation method, first check whether the FB memory is configured from mmap.ini, then check /config/fbdev.ini, and finally allocate it by default according to the fbdev driver.





1.6. MMAP adjustment method

1.6.1 mmap.ini description

The built-in DDR2 capacity of the chip is 64M, we use mmap.ini as the memory configuration file, which contains the pre-allocated memory, This part of the memory linux system is invisible in the linux management.

The /config/mmap.ini in the compiled image comes from /project/board/i2m/mmap, specifically which header file in the project to use, Then you need to check /project/configs/nvr/i2m/8.2.1/nor.glibc-squashfs.011a.64: For example,

MMAP = MMAP_I2M_64M.h If it is SSD202, you need to modify it to MMAP =

MMAP_I2M_128M.h Linux will parse mmap. ini to initialize the memory part, including

the memory reserved by sz such as ai, ao, etc. of the layout in mmap, which can be accessed from # cat /

sys/kernel/debug/memblock/reserved

```
/ # cat /sys/kernel/debug/memblock/reserved
0: 0x20004000..0x20007fff
1: 0x20008240..0x20469637
2: 0x22000000..0x221fffff
3: 0x22265000..0x2228cfff
4: 0x2228f9c0..0x222fefff
5: 0x222ff440..0x222ff87b
6: 0x222ff880..0x222ff8bb
7: 0x222ff8c0..0x222ff937
8: 0x222ff940..0x222ff947
9: 0x222ff980..0x222ff987
10: 0x222ff9c0..0x222ff9c3
11: 0x222ffa00..0x222ffa03
12: 0x222ffa40..0x222ffba4
13: 0x222ffb00..0x222ffd24
14: 0x222ffd40..0x222ffea4
15: 0x222ffeac..0x222ffec6
16: 0x222ffec8..0x222ffee2
17: 0x222ffee4..0x222ffefe
18: 0x222fff00..0x222fff1f
19: 0x222fff24..0x222fff3e
20: 0x222fff40..0x222fff4b
21: 0x222fff60..0x237fffff
```

1.6.2 Adjust Layout in mmap.ini

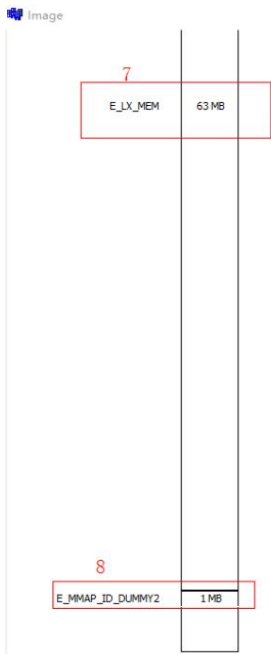
Open the SCA.exe software and load the configuration file, as

shown in the figure below: 1. Click to load the corresponding configuration file, such as: /project/board/i2m/mmap/MMAP_I2M_64M.h 2. After modification, click "save" to save the current modification 3. Select the corresponding

layout block, then modify its size to modify, click "modify" to complete the editing of the selected block 4. The layout memory block that has been edited in the current mmap.ini MB, and then enter the corresponding capacity in the input box. 6. The name of the currently selected layout block can also be edited. For example, if you add a layout block, you can enter a custom name in this area. 7. The current mmap.ini configuration of LX Length 8. The length of DUMMY in the current mmap.ini configuration. Dummy contains more small blocks of memory. 9. Memory less than 1M is attached to the dummy layout block.



{SigmaStar Display}
{Smart Display}
{SSD201+1.0}



9

E_MMAP_ID_CMDQ	128 KB
E_MMAP_ID_GE_VQ	128 KB
E_MMAP_ID_AO	300 KB
E_MMAP_ID_AI	300 KB

MMAP | Register Diagnose | Protection Tool

System: [Dropdown]
Chip: [Dropdown]
MIU: [Dropdown] 64
CPU Align: 4K
IP: E_MMAP_ID_AO (labeled 6)
Address: 0x0003E40000
Size: 307200 (labeled 5) (0x00004b000)
Alignment: 4096 (0x000001000)
Memory Type: TYPE_NONE [UNCACHE]
CMA Type: TYPE_NONE
Comment: [Text Area]
MIU 0: [List Box] E_LX_MEM, E_MMAP_ID_DUMMY2, E_MMAP_ID_CMDQ, E_MMAP_ID_GE_VQ, E_MMAP_ID_AO (labeled 4), E_MMAP_ID_AI
MIU 1: [List Box]
Buttons: Add, Load (labeled 1), Delete, Save (labeled 2), Insert, Save Image, Show Image, Rule, Close, Modify (labeled 3), Bandwidth, Show cma, MIU Setting, Rule Editor

1.7. Notes

none